

LAPORAN PRATIKUM

STRUKTUR DATA

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira Putri



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang,

Muhammad Yasin Habiburrahman

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1	Error! Bookmark not defined.
BAB III KESIMPULAN.....	8
DAFTAR PUSTAKA.....	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum ini dipelajari tiga algoritma sorting lanjutan yaitu Merge Sort, Quick Sort, dan Shell Sort. Merge Sort bekerja dengan strategi divide and conquer membagi array secara rekursif menjadi dua bagian hingga setiap bagian hanya berisi satu elemen, lalu menggabungkannya kembali secara terurut. Quick Sort juga menggunakan strategi divide and conquer dengan memilih sebuah elemen sebagai pivot lalu mempartisi array sehingga elemen yang lebih kecil dari pivot berada di kiri dan yang lebih besar di kanan, diulang secara rekursif.

Selain mempelajari algoritmanya secara konsol, praktikum ini juga mengimplementasikan Bubble Sort dan Merge Sort dalam bentuk aplikasi GUI menggunakan Java Swing dengan visualisasi warna untuk menandai elemen yang sedang dibandingkan dan ditukar, sehingga proses sorting dapat dipahami secara lebih intuitif dan interaktif.

1.2 Tujuan

1. Memahami konsep dan cara kerja algoritma Merge Sort, Quick Sort, dan Shell Sort sebagai algoritma sorting lanjutan.
2. Mampu mengimplementasikan ketiga algoritma sorting lanjutan tersebut menggunakan bahasa pemrograman Java.
3. Mampu membangun aplikasi GUI menggunakan Java Swing dengan penambahan visualisasi warna untuk menandai elemen yang sedang diproses.

1.3 Manfaat Praktikum

1. Praktikan mampu membandingkan efisiensi algoritma sorting dasar dan lanjutan, serta menentukan algoritma yang tepat sesuai kebutuhan dan karakteristik data.
2. Praktikan memahami strategi divide and conquer sebagai pendekatan pemecahan masalah yang efisien, yang tidak hanya terbatas pada sorting tetapi juga dapat diterapkan pada berbagai permasalahan pemrograman lainnya.
3. Praktikan mendapatkan pengalaman membangun GUI yang lebih interaktif dengan penambahan visualisasi warna sebagai umpan balik visual terhadap proses yang sedang berjalan.

BAB II

PEMBAHASAN

2.1 Shell Sort

```
public static void ShellSort_2016(int[] A_2016) {  
    int n_2016 = A_2016.length;  
    int gap_2016 = n_2016/2;  
    while(gap_2016 > 0) {  
        for(int i_2016=gap_2016 ; i_2016<n_2016; i_2016++) {  
            int temp_2016 = A_2016[i_2016];  
            int j_2016=i_2016;  
            while(j_2016>=gap_2016 && A_2016[j_2016-gap_2016] > temp_2016) {  
                A_2016[j_2016]= A_2016[j_2016-gap_2016];  
                j_2016= j_2016-gap_2016;  
            }  
            A_2016[j_2016] = temp_2016;  
        }  
        gap_2016 = gap_2016/2;  
    }  
}
```

Kode Program 2.1 – Blok Program Shell Sort

Pada insertion sort biasa, elemen hanya bisa bergeser satu posisi per langkah sehingga lambat untuk elemen yang jauh dari posisi akhirnya. Shell Sort mengatasinya dengan membandingkan elemen yang berjarak gap terlebih dahulu sebelum memperkecil gap secara bertahap. Gap diinisialisasi sebagai $n/2$ dan terus dibagi dua setiap iterasi loop luar hingga mencapai 0. Untuk setiap nilai gap, dilakukan proses yang mirip Insertion Sort. Temp menyimpan elemen di indeks i , lalu loop while menggeser elemen yang lebih besar dari temp sejauh gap ke kanan. Setelah tidak ada lagi elemen yang lebih besar, temp ditempatkan di posisi yang tepat. Ketika gap akhirnya bernilai 1, Shell Sort berperilaku identik dengan Insertion Sort biasa. Namun karena elemen-elemen sudah hampir terurut dari iterasi gap sebelumnya, prosesnya jauh lebih cepat.

Misalkan dilakukan simulasi sorting untuk array {3,10,4,6,8,9,7,2,1,5}

Gap	Hasil
5	3, 9, 4, 6, 8, 10, 7, 2, 1, 5 → 3, 2, 1, 5, 8, 9, 7, 4, 6, 10
2	1, 2, 3, 4, 6, 5, 7, 9, 8, 10
1	1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Tabel 2.1 – Tabel Shell Sort

2.2 Quick Sort

```

public class QuickSort_2511532016 {
    static void Swap_2016(int[] arr_2016, int i_2016, int j_2016) {
        int temp_2016 = arr_2016[i_2016];
        arr_2016[i_2016] = arr_2016[j_2016];
        arr_2016[j_2016] = temp_2016;
    }
    static void MedianOfThree_2016(int[] arr_2016, int low_2016, int high_2016) {
        int mid_2016 = low_2016 + (high_2016-low_2016)/2;

        if(arr_2016[low_2016] > arr_2016[mid_2016]) {
            Swap_2016(arr_2016, high_2016, mid_2016);
        }
        if(arr_2016[low_2016] > arr_2016[high_2016]) {
            Swap_2016(arr_2016, high_2016, mid_2016);
        }
        if(arr_2016[mid_2016] > arr_2016[high_2016]) {
            Swap_2016(arr_2016, high_2016, mid_2016);
        }
        Swap_2016(arr_2016, high_2016, mid_2016);
    }

    static int Partition_2016(int[] arr_2016, int low_2016, int high_2016) {
        MedianOfThree_2016(arr_2016, low_2016, high_2016);

        int pivot_2016 = arr_2016[high_2016];
        int i_2016 = low_2016-1;

        for(int j_2016=low_2016; j_2016<=high_2016-1;j_2016++ ) {
            if(arr_2016[j_2016]<pivot_2016) {
                i_2016++;
                Swap_2016(arr_2016, i_2016, j_2016);
            }
        }
        Swap_2016(arr_2016, 1+i_2016, high_2016);
        return i_2016+1;
    }

    static void QuickSort_2016(int[]arr_2016,int low_2016, int high_2016) {
        if(low_2016<high_2016) {
            int pi_2016= Partition_2016(arr_2016,low_2016,high_2016);
            QuickSort_2016(arr_2016, low_2016, pi_2016-1);
            QuickSort_2016(arr_2016,pi_2016+1, high_2016);
        }
    }
}

```

Kode Program 2.2 – Blok Program Quick Sort

2.2.1 Swap

Method helper sederhana untuk menukar dua elemen array di indeks i dan j menggunakan variabel sementara temp. Dipakai berulang kali oleh method lainnya.

2.2.2 Median Of Three

Sederhananya, method ini memilih tiga nilai dari nilai indeks pertama, terakhir dan di tengah. Kemudian nilai median akan diletakkan ke indeks paling akhir.

2.2.3 Partition

Pertama MedianOfThree dipanggil untuk menentukan pivot di posisi high. Kemudian i diinisialisasi ke low-1 sebagai penanda batas kiri partisi. Loop for menelusuri array dari low sampai high-1. Setiap elemen yang lebih kecil dari pivot ditukar ke sisi kiri dengan menginkremen i terlebih dahulu lalu menukar arr[i] dengan arr[j]. Setelah loop selesai, pivot ditempatkan di posisi akhirnya yaitu i+1 dengan menukar arr[i+1] dan arr[high]. Indeks i+1 dikembalikan sebagai posisi pivot.

2.2.4 Algoritma QuickSort

Method rekursif utama. Jika low < high, Partition dipanggil untuk mendapatkan posisi pivot pi. Array kemudian dibagi dua, sisi kiri [low, pi-1] dan sisi kanan [pi+1, high], dan QuickSort dipanggil secara rekursif pada keduanya. Pivot tidak ikut diproses lagi karena sudah berada di posisi akhirnya.

2.3 Merge Sort

```
public class MergeSort_201522000 {
    static void Merge(int[] arr, int l_2016, int m_2016, int r_2016) {
        int m1_2016 = m_2016 - l_2016 + 1;
        int m2_2016 = r_2016 - m_2016;

        int l_2016[] = new int[m1_2016];
        int m_2016[] = new int[m2_2016];

        for (int i_2016 = 0; i_2016 < m1_2016; i_2016++) l_2016[i_2016] = arr[l_2016 + i_2016];
        for (int j_2016 = 0; j_2016 < m2_2016; j_2016++) m_2016[j_2016] = arr[m_2016 + j_2016];
        int i_2016 = 0, j_2016 = 0;

        int k_2016 = l_2016;
        while (i_2016 < m1_2016 && j_2016 < m2_2016) {
            if (l_2016[i_2016] <= m_2016[j_2016]) {
                arr[k_2016] = l_2016[i_2016];
                i_2016++;
            } else {
                arr[k_2016] = m_2016[j_2016];
                j_2016++;
            }
            k_2016++;
        }
        while (i_2016 < m1_2016) {
            arr[k_2016] = l_2016[i_2016];
            i_2016++;
            k_2016++;
        }
        while (j_2016 < m2_2016) {
            arr[k_2016] = m_2016[j_2016];
            j_2016++;
            k_2016++;
        }
    }

    public static void printArray(int[] arr) {
        for (int i_2016 : arr) System.out.print(i_2016 + " ");
        System.out.println();
    }

    void sort_2016(int arr[], int l, int r) {
        if (l < r) {
            int m = (l + r) / 2;
            sort_2016(arr, l, m);
            sort_2016(arr, m + 1, r);
            Merge(arr, l, m, r);
        }
    }
}
```

Kode Program 2.3 – Blok Program Merge Sort

2.3.1 Method Merge

```
public class MergeSort_2511532016 {
    static void Merge(int[] arr, int l_2016, int m_2016, int r_2016) {
        int n1_2016 = m_2016 - l_2016 + 1;
        int n2_2016 = r_2016 - m_2016;

        int L_2016[] = new int[n1_2016];
        int R_2016[] = new int[n2_2016];

        for(int i_2016=0; i_2016<n1_2016; i_2016++) L_2016[i_2016] = arr[i_2016+l_2016];
        for(int j_2016=0; j_2016<n2_2016; j_2016++) R_2016[j_2016] = arr[m_2016+1+j_2016];
        int i_2016=0, j_2016=0;

        int k_2016=l_2016;
        while(i_2016<n1_2016 && j_2016<n2_2016) {
            if(L_2016[i_2016] <= R_2016[j_2016]) {
                arr[k_2016] = L_2016[i_2016];
                i_2016++;
            } else {
                arr[k_2016] = R_2016[j_2016];
                j_2016++;
            }
            k_2016++;
        }
        while(i_2016<n1_2016) {
            arr[k_2016] = L_2016[i_2016];
            i_2016++;
            k_2016++;
        }
        while(j_2016<n2_2016) {
            arr[k_2016] = R_2016[j_2016];
            j_2016++;
            k_2016++;
        }
    }
}
```

Kode Program 2.4 – Blok Program Merge untuk Menggabungkan Sub-array

Method ini menggabungkan dua sub-array yang sudah terurut menjadi satu array terurut. Pertama dihitung ukuran subarray kiri $n1 = m-l+1$ dan kanan $n2 = r-m$, lalu dibuat dua array sementara L dan R yang masing-masing diisi dengan elemen subarray kiri dan kanan dari array asli.

Kemudian tiga pointer digunakan — i untuk menelusuri L, j untuk menelusuri R, dan k sebagai posisi penulisan ke array asli. Loop while pertama membandingkan elemen $L[i]$ dan $R[j]$, lalu menyalin yang lebih kecil ke $arr[k]$. Loop while kedua dan ketiga menyalin sisa elemen yang belum tersalin dari L atau R jika salah satunya habis lebih dahulu.

2.3.2 Method Sort

```
void sort_2016(int arr[], int l, int r) {
    if(l<r) {
        int m = (l+r)/2;
        sort_2016(arr, l, m);
        sort_2016(arr, m+1, r);
        Merge(arr, l, m, r);
    }
}
```

Kode Program 2.5 – Blok Program Sort

Method rekursif yang mengimplementasikan strategi *divide and conquer*. Jika $l < r$, array dibagi dua di titik tengah $m = (l+r)/2$. sort dipanggil rekursif untuk sisi kiri $[l, m]$ dan sisi kanan $[m+1, r]$, lalu Merge dipanggil untuk menggabungkan keduanya.

Rekursi berhenti ketika subarray hanya berisi satu elemen karena kondisi $l < r$ tidak terpenuhi.

2.4 GUI Bubble Sort

2.4.1 Atribut dan Konstruktor

```
public class BubbleSort_GUI2511532016 extends JFrame {
    private static final long serialVersionUID = 1L;

    private int[] array_2016;
    private JLabel[] labelArray_2016;
    private JButton stepButton_2016, resetButton_2016, setButton_2016;
    private JTextField inputField_2016;
    private JPanel panelArray_2016;
    private JTextArea stepArea_2016;
    private int i_2016 = 1, j_2016;
    private boolean sorting_2016 = false;
    private int stepCount_2016 = 1;

    public BubbleSort_GUI2511532016() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        //Panel Input
        JPanel inputPanel_2016 = new JPanel(new FlowLayout());
        inputField_2016 = new JTextField(30);
        setButton_2016 = new JButton("Set Array");
        inputPanel_2016.add(new JLabel("Masukkan angka (pisahkan dengan koma)"));
        inputPanel_2016.add(inputField_2016);
        inputPanel_2016.add(setButton_2016);

        //Panel Array visual
        panelArray_2016 = new JPanel();
        panelArray_2016.setLayout(new FlowLayout());

        //Panel Kontrol
        JPanel controlPanel_2016 = new JPanel();
        stepButton_2016 = new JButton("Langkah selanjutnya");
        resetButton_2016 = new JButton("Reset");
        stepButton_2016.setEnabled(false);
        controlPanel_2016.add(stepButton_2016);
        controlPanel_2016.add(resetButton_2016);

        //Area teks untuk log langkah-langkah
        stepArea_2016 = new JTextArea(8, 60);
        stepArea_2016.setEditable(false);
        stepArea_2016.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane_2016 = new JScrollPane(stepArea_2016);

        //Tambahkan panel ke frame
        add(inputPanel_2016, BorderLayout.NORTH);
        add(panelArray_2016, BorderLayout.CENTER);
        add(controlPanel_2016, BorderLayout.SOUTH);
        add(scrollPane_2016, BorderLayout.EAST);

        //Event set array
        setButton_2016.addActionListener( e-> setArrayFromInput_2016());

        //Event set array
        stepButton_2016.addActionListener( e-> performStep());

        //Event set array
        resetButton_2016.addActionListener( e-> resetHighlights_2016());
    }
}
```

Kode Program 2.6 – Blok Program Atribut dan Konstruktor pada Program GUI Bubble Sort

Struktur atribut dan konstruktor GUI Bubble Sort pada dasarnya identik dengan GUI Insertion Sort yang telah dibahas sebelumnya. Perbedaannya terletak pada inisialisasi i dan j yang keduanya dimulai dari 0, karena Bubble Sort membutuhkan dua indeks yang keduanya aktif sejak awal, berbeda dengan Insertion Sort yang hanya membutuhkan i dimulai dari 1. Selain itu setiap JLabel ditambahkan `setOpaque(true)` agar background color bisa diubah untuk keperluan visualisasi warna.

2.4.2 Memasukkan Nilai untuk Jendela GUI

```
private void setArrayFromInput_2016() {
    String text_2016 = InputField_2016.getText().trim();
    if(text_2016.isEmpty()) return;
    String[] parts_2016 = text_2016.split(",");
    array_2016 = new int[parts_2016.length];
    try {
        for(int k_2016=0; k_2016< parts_2016.length; k_2016++) {
            array_2016[k_2016] = Integer.parseInt(parts_2016[k_2016].trim());
        }
        catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukan hanya angka yang dipisahkan dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }
    i_2016=0;
    j_2016=0;
    stepCount_2016 = 1;
    sorting_2016 = true;
    stepButton_2016.setEnabled(true);
    stepArea_2016.setText("");
    panelArray_2016.removeAll();
    labelArray_2016 = new JLabel[array_2016.length];
    for(int k_2016=0; k_2016<array_2016.length; k_2016++) {
        labelArray_2016[k_2016] = new JLabel(String.valueOf(array_2016[k_2016]));
        labelArray_2016[k_2016].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_2016[k_2016].setOpaque(true);
        labelArray_2016[k_2016].setBackground(Color.WHITE);
        labelArray_2016[k_2016].setBorder(BorderFactory.createLineBorder(Color.black));
        labelArray_2016[k_2016].setPreferredSize(new Dimension(50,50));
        labelArray_2016[k_2016].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_2016.add(labelArray_2016[k_2016]);
    }
    panelArray_2016.revalidate();
    panelArray_2016.repaint();
}
```

Kode Program 2.7 – Blok Program

Logikanya sama dengan versi Insertion Sort. Perbedaannya `i_2016` dan `j_2016` keduanya direset ke 0, dan setiap label ditambahkan `setBackground(Color.WHITE)` sebagai warna default sebelum sorting dimulai.

2.4.3 Mengurutkan Setiap Penekanan Tombol

```
private void performStep_2016() {
    if (!sorting_2016 || i_2016 >= array_2016.length - 1) {
        sorting_2016 = false;
        stepButton_2016.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }
    StringBuilder stepLog_2016 = new StringBuilder();
    labelArray_2016[j_2016].setBackground(Color.CYAN);
    labelArray_2016[j_2016 + 1].setBackground(Color.CYAN);
    if (array_2016[j_2016] > array_2016[j_2016 + 1]) {
        int temp_2016 = array_2016[j_2016];
        array_2016[j_2016] = array_2016[j_2016 + 1];
        array_2016[j_2016 + 1] = temp_2016;
        labelArray_2016[j_2016].setBackground(Color.RED);
        labelArray_2016[j_2016 + 1].setBackground(Color.RED);
        stepLog_2016.append("Langkah ").append(stepCount_2016).append(": Menukar elemen ke-")
            .append(j_2016).append(" ").append(array_2016[j_2016 + 1]).append(" dengan elemen ke-")
            .append(j_2016 + 1).append(" ").append(array_2016[j_2016]).append("\n");
    } else {
        stepLog_2016.append("Langkah ").append(stepCount_2016).append(": Tidak ada pertukaran antara ke-")
            .append(j_2016).append(" dan ke-").append(j_2016 + 1).append("\n");
    }
    stepLog_2016.append("Hasil: ").append(arrayToString_2016(array_2016)).append("\n\n");
    stepArea_2016.append(stepLog_2016.toString());
    updateLabels_2016();
    j_2016++;
    if (j_2016 >= array_2016.length - i_2016 - 1) {
        j_2016 = 0;
        i_2016++;
    }
    stepCount_2016++;
    if (i_2016 >= array_2016.length - 1) {
        sorting_2016 = false;
        stepButton_2016.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}
```

Kode Program 2.8 – Blok Program

Setiap klik tombol hanya memproses satu perbandingan antara `arr[j]` dan `arr[j+1]`. Jika terjadi pertukaran warnanya berubah menjadi merah sebagai penanda visual. Log mencatat apakah terjadi pertukaran atau tidak pada langkah tersebut.

Setelah satu perbandingan selesai, `j` dinaikkan. Jika `j` sudah mencapai batas `n-i-1`, berarti satu pass selesai. `j` direset ke 0 dan `i` dinaikkan untuk pass berikutnya. Jika `i` sudah mencapai `n-1`, sorting selesai dan dialog ditampilkan.

2.4.4 Mengupdate Label

```
private void updateLabels_2016() {
    for(int k_2016=0; k_2016<array_2016.length; k_2016++) {
        labelArray_2016[k_2016].setText(String.valueOf(array_2016[k_2016]));
    }
}
```

Kode Program 2.9 – Blok Program

Berfungsi memperbarui teks label visual dan mengkonversi array ke String untuk keperluan log.

2.4.5 Mereset GUI

```
private void resetHighlights_2016() {
    inputField_2016.setText("");
    panelArray_2016.removeAll();
    panelArray_2016.revalidate();
    panelArray_2016.repaint();
    stepArea_2016.setText("");
    stepButton_2016.setEnabled(false);
    sorting_2016 = false;
    i_2016=0;
    j_2016=0;
    stepCount_2016 = 1;
}
```

Kode Program 2.10 – Blok Program

`resetHighlights_2016()` mengembalikan warna semua label ke putih tanpa mengubah data atau state sorting, dipanggil di awal setiap `performStep` agar highlight langkah sebelumnya hilang sebelum highlight baru ditampilkan. Reset penuh dilakukan oleh method yang juga bernama `resetHighlights_2016()` yang dipanggil oleh tombol Reset. Method ini mengosongkan seluruh komponen dan mereset semua variabel ke kondisi awal

BAB III

LAPORAN TUGAS PRAKTIKUM

3.1 Shell Sort

Pada tugas praktikum kali ini, diberikan beberapa opsi sorting yang akan digunakan untuk mengurutkan paling banyak 7 data lagu. Algoritma sorting yang digunakan yaitu antara shell, quick, dan merge sort.

Tugas ini akan mengimplementasikan shell sort karena lebih mudah dipahami secara intuitif dibandingkan dua algoritma sort lainnya yang menggunakan rekursi untuk melakukan proses sorting.

Kompleksitas yang dimiliki oleh shell sort adalah $O(n^2)$ untuk kasus terburuk, dan $O(n \log(n))$ pada kasus terbaiknya. Karena data yang diurutkan relatif sedikit, hal ini tidak terlalu dipermasalahkan untuk kasus terburuk.

3.2 ADT Lagu

```
public class Lagu_2511532016 {
    private String judul_2016;
    private String penyanyi_2016;
    private int durasi_2016;

    public Lagu_2511532016(String judul_2016, String penyanyi_2016, int durasi_2016) {
        this.judul_2016 = judul_2016;
        this.penyanyi_2016 = penyanyi_2016;
        this.durasi_2016 = durasi_2016;
    }

    public String getJudul_2016() { return judul_2016; }
    public String getPenyanyi_2016() { return penyanyi_2016; }
    public int getDurasi_2016() { return durasi_2016; }

    public void setJudul_2016(String judul_2016) { this.judul_2016 = judul_2016; }
    public void setPenyanyi_2016(String penyanyi_2016) { this.penyanyi_2016 = penyanyi_2016; }
    public void setDurasi_2016(int durasi_2016) { this.durasi_2016 = durasi_2016; }

    @Override
    public String toString() {
        return judul_2016 + " - " + penyanyi_2016 + " (" + durasi_2016 + " detik)";
    }
}
```

Kode Program 3.1 – Blok Program ADT Lagu

ADT ini memiliki atribut judul, penyanyi, dan durasi. ADT ini juga memiliki konstruktor, beserta mutator dan accessor untuk masing-masing atributnya. Dia juga memiliki fungsi `toString()` yang digunakan untuk menampilkan atribut-atribut dari suatu lagu secara lengkap.

3.2 Input Data

```
private Lagu_2511532016[] dataLagu_2016 = new Lagu_2511532016[100];
private int jumlahLagu_2016 = 0;
public void inputData_2016() {
    dataLagu_2016[0] = new Lagu_2511532016("Wind's Anthem", "Eve", 223);
    dataLagu_2016[1] = new Lagu_2511532016("spiral", "LONGMAN", 232);
    dataLagu_2016[2] = new Lagu_2511532016("Souvenir", "Bump of Chicken", 265);
    dataLagu_2016[3] = new Lagu_2511532016("Same Blue", "OFFICIAL HIGE DANDISM", 238);
    dataLagu_2016[4] = new Lagu_2511532016("Kaiju", "sakanaction", 253);
    dataLagu_2016[5] = new Lagu_2511532016("Hello, World!", "Bump of Chicken", 249);
    dataLagu_2016[6] = new Lagu_2511532016("Beneath the Mask", "Lyn", 278);
    dataLagu_2016[7] = new Lagu_2511532016("Absolute Zero", "natori", 199);
    dataLagu_2016[8] = new Lagu_2511532016("Memories of You", "Azumi Takahashi", 400);
    dataLagu_2016[9] = new Lagu_2511532016("SPECIALZ", "King Gnu", 240);
    jumlahLagu_2016 = 10;
}
```

Kode Program 3.2 – Blok Program yang Berisikan Data Lagu-Lagu

Method ini hanya berisikan data-data yang akan disort secara leksikografis nantinya

3.3 Kelas Sorting

```
public void shellSort_2016() {
    int n_2016 = jumlahLagu_2016;
    int gap_2016 = n_2016 / 2;
    while (gap_2016 > 0) {
        for (int i_2016 = gap_2016; i_2016 < n_2016; i_2016++) {
            Lagu_2511532016 temp_2016 = dataLagu_2016[i_2016];
            int j_2016 = i_2016;
            while (j_2016 >= gap_2016 &&
                dataLagu_2016[j_2016 - gap_2016].getJudul_2016().
                    compareToIgnoreCase(temp_2016.getJudul_2016()) > 0) {
                dataLagu_2016[j_2016] = dataLagu_2016[j_2016 - gap_2016];
                j_2016 -= gap_2016;
            }
            dataLagu_2016[j_2016] = temp_2016;
        }
        gap_2016 /= 2;
    }
}
```

Kode Program 3.3 – Blok Program Shell Sort

Pertama diinisialisasikan sebuah variabel n untuk menyimpan banyak lagu. Pada kasus ini bernilai 10. Kemudian gap yang pada awalnya bernilai $10/2=5$. Lalu while loop dijalankan selagi nilai gap masih di atas 0. Urutan gap adalah $5 \rightarrow 2 \rightarrow 1 \rightarrow 0$. Saat gap bernilai 1, algoritma akan berubah menjadi insertion sort.

For loop pertama digunakan untuk melakukan perulangan pada elemen yang akan disisipkan. Pada awalnya gap bernilai 5, maka $i=5$. Kemudian elemen akan disimpan pada variabel sementara. Selanjutnya variabel j akan menyimpan nilai yang sama dengan i.

While loop selanjutnya akan berjalan selagi masih ada elemen pada sebelah kiri gap, yang ditandai dengan $j \geq \text{gap}$, dan judul lagu pada indeks ke- $(j-\text{gap})$ memiliki nilai leksikografis yang lebih besar daripada indeks ke-i. Perbandingan judul lagu dilakukan dengan tidak memperhatikan huruf kapital.

Apabila judul lagu di sebelah kiri secara alfabet lebih besar daripada judul lagu yang sedang diproses, maka lagu tersebut digeser ke kanan sejauh satu gap. Setelah pergeseran dilakukan, program terus bergerak ke kiri dengan jarak yang sama dan kembali melakukan perbandingan. Proses ini berulang sampai ditemukan posisi yang sesuai atau sampai tidak ada lagi elemen yang dapat dibandingkan.

Setelah posisi yang tepat ditemukan, lagu yang sebelumnya disimpan dalam temp ditempatkan pada posisi tersebut. Maka elemen tersebut telah berada pada urutan yang lebih benar dibandingkan sebelumnya.

Ketika seluruh data telah diproses untuk nilai gap saat itu, nilai gap dibagi dua sehingga jarak perbandingan menjadi lebih kecil. Pengurutan kemudian diulang kembali dengan jarak yang baru. Semakin kecil nilai gap, semakin detail proses pengurutannya.

Pada akhirnya, ketika nilai gap menjadi 1, algoritma bekerja seperti Insertion Sort biasa. Namun karena data sudah sebagian besar tertata oleh proses sebelumnya, jumlah pergeseran yang diperlukan menjadi lebih sedikit. Setelah proses dengan gap = 1 selesai dan gap berubah menjadi 0, seluruh data lagu telah terurut berdasarkan judul secara alfabetis dan algoritma berakhir.

3.4 Hasil Sorting

```

=== Sorting Playlist NIM: 2511532016 ===
Algoritma: Shell Sort (Judul A-Z)

Data Sebelum Sorting:
1. Wind's Anthem - Eve (223 detik)
2. spiral - LONGMAN (232 detik)
3. Souvenir - Bump of Chicken (265 detik)
4. Same Blue - OFFICIAL HIGE DANDISM (238 detik)
5. Kaiju - sakanaction (253 detik)
6. Hello, World! - Bump of Chicken (249 detik)
7. Beneath the Mask - Lyn (278 detik)
8. Absolute Zero - natori (199 detik)
9. Memories of You - Azumi Takahashi (400 detik)
10. SPECIALZ - King Gnu (240 detik)

Data Setelah Shell Sort (Judul A-Z):
1. Absolute Zero - natori (199 detik)
2. Beneath the Mask - Lyn (278 detik)
3. Hello, World! - Bump of Chicken (249 detik)
4. Kaiju - sakanaction (253 detik)
5. Memories of You - Azumi Takahashi (400 detik)
6. Same Blue - OFFICIAL HIGE DANDISM (238 detik)
7. Souvenir - Bump of Chicken (265 detik)
8. SPECIALZ - King Gnu (240 detik)
9. spiral - LONGMAN (232 detik)
10. Wind's Anthem - Eve (223 detik)

```

Gambar 3.1 – Hasil Setelah Sorting

BAB IV

KESIMPULAN

Pada praktikum ini telah berhasil diimplementasikan algoritma Shell Sort untuk mengurutkan data lagu berdasarkan judul secara alfabetis menggunakan bahasa pemrograman Java. Algoritma Shell Sort bekerja dengan membandingkan elemen yang berjarak tertentu (gap) dan secara bertahap memperkecil jarak tersebut hingga menjadi satu, sehingga proses pengurutan menjadi lebih efisien dibandingkan Insertion Sort biasa pada banyak kasus.

Melalui praktikum ini, dapat dipahami bahwa Shell Sort merupakan pengembangan dari Insertion Sort yang mampu mengurangi jumlah pergeseran elemen dengan melakukan pengurutan awal pada elemen-elemen yang berjauhan. Implementasi yang dilakukan berhasil mengurutkan data lagu berdasarkan urutan leksikografis tanpa memperhatikan perbedaan huruf kapital dan huruf kecil.

DAFTAR PUSTAKA

- [1]Oracle. *Java Documentation*. <https://docs.oracle.com/javase/>